DINO

We use the pretrained DINO ViT-S/16 vision model.

We can understand our retrieval workflow as a pipeline.

01 Split image into patches

02 Patch embedding

03 Add CLS token

04 Transformer processing

05 Feature output

06 Image retrieval

Source: Caron et al., ICCV 2021

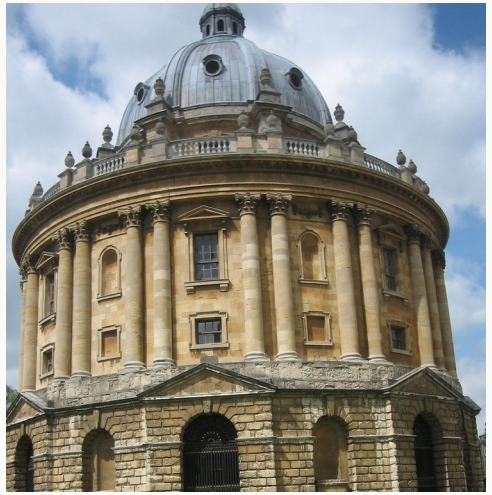
Emerging Properties in Self-Supervised Vision Transformers

Dataset: Revisited Oxford

Oxford building photos with revised retrieval annotations.
The DINO paper also uses this benchmark.

Query

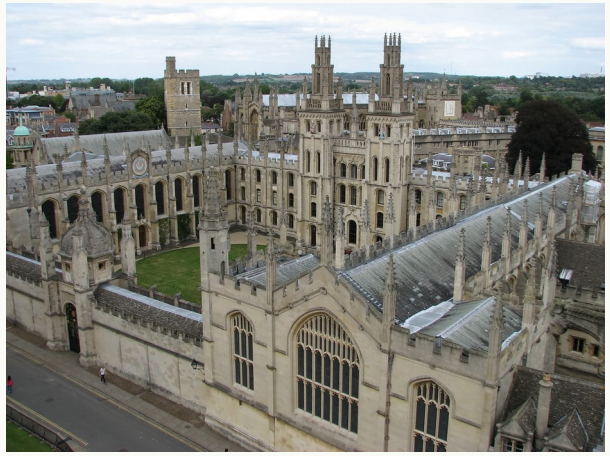
70 test searches



Gallery

4,993 images to search





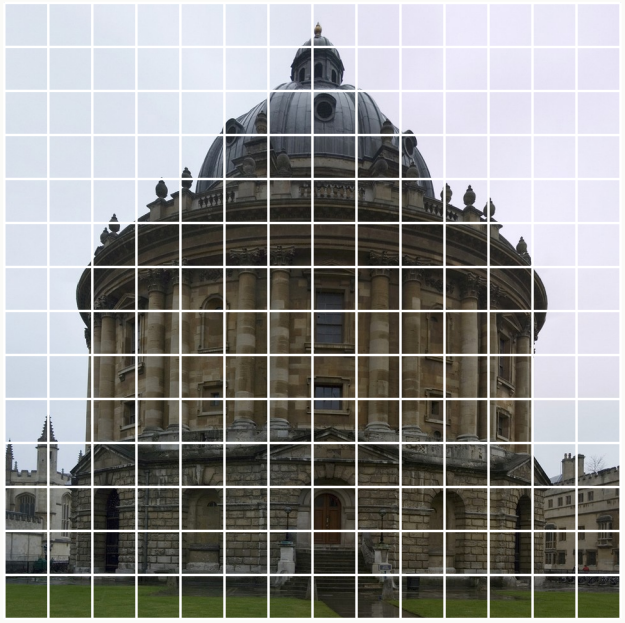
Sources: Oxford Buildings dataset; Radenović et al., CVPR 2018

1–2. Patches and patch embedding

A real image from our gallery



224 × 224 pixels



14 × 14 = 196 patches

p_1
p_2
⋮
p_{196}

196 vectors

384 numbers each

$F(\text{patch}_i) = p_i$

3. Add CLS token

Each patch vector has 384 numbers. The CLS vector c does too.



One fixed CLS vector

196 patch vectors from this image

The same starting c goes into every image.

c = (0.030054, −0.000030, −0.000129, −0.002799, …)

4–5. Transformer and feature output

A learned function processes the vectors together.



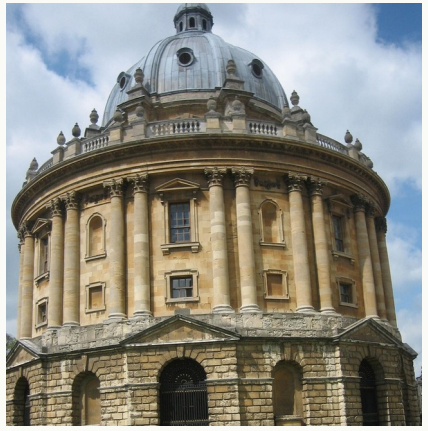
z A compact summary of the whole image.

x_i A patch feature that also uses information from other patches.

6. Image retrieval with CLS

Compare the query vector with every gallery vector, then rank.

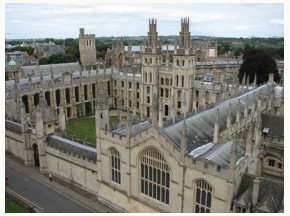
Query



$\rightarrow$ z_Q $\rightarrow$ cosine $\leftarrow$ z_I $\leftarrow$

Gallery





Each of 4,993 images

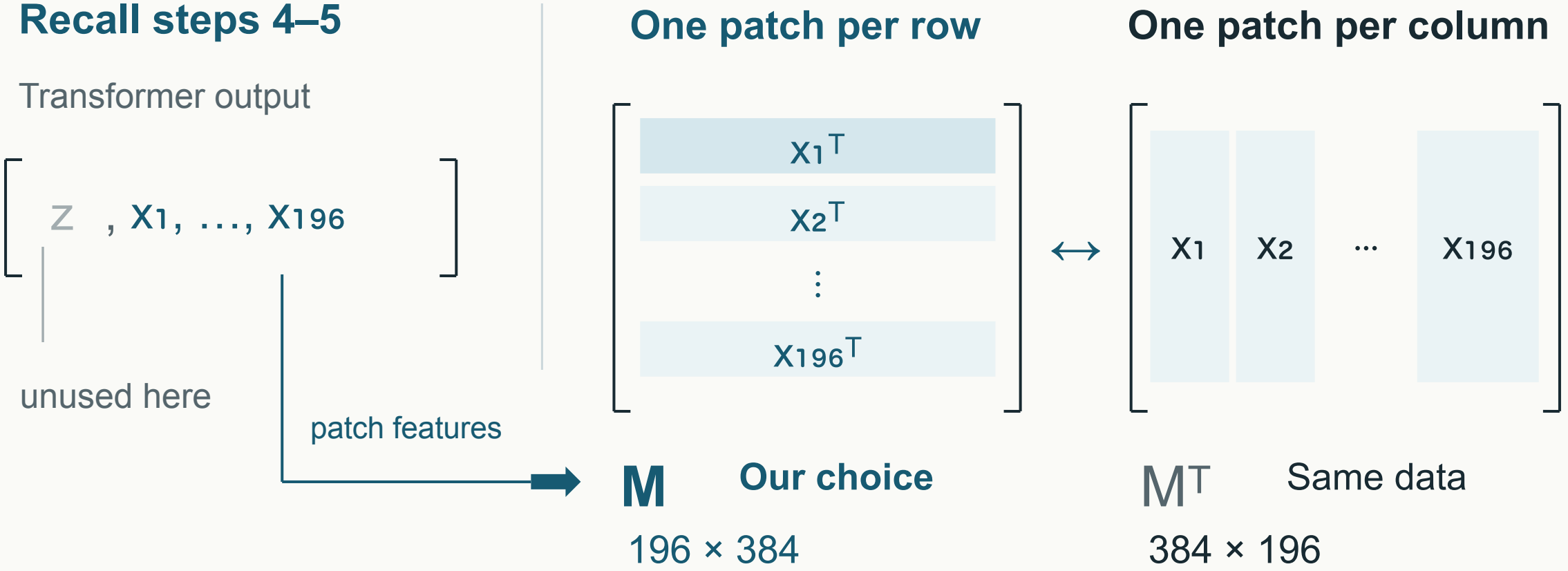
$$\cos(z_Q, z_I) = \frac{z_Q^T z_I}{\|z_Q\|_2 \|z_I\|_2}$$

Closer to 1 = more similar directions in the learned feature space.

Open interactive retrieval demo

Patch matrix M

We use the patch features and leave z out of this comparison.



SVD and Gram matrix

By the SVD theorem, every real m × n matrix M has a factorization:

$$M = U \Sigma V^{\mathrm{T}}$$

Gram matrix: inner products between columns

$$G = U^{\mathrm{T}} U = \begin{bmatrix} u_1^{\mathrm{T}} u_1 & u_1^{\mathrm{T}} u_2 & \cdots & u_1^{\mathrm{T}} u_m \\ u_2^{\mathrm{T}} u_1 & u_2^{\mathrm{T}} u_2 & \cdots & u_2^{\mathrm{T}} u_m \\ \vdots & \vdots & \ddots & \vdots \\ u_m^{\mathrm{T}} u_1 & u_m^{\mathrm{T}} u_2 & \cdots & u_m^{\mathrm{T}} u_m \end{bmatrix}$$

The columns of U

$$U \in \mathbb{R}^{m \times m}$$

$$U = [u_1, u_2, \ldots, u_m]$$

u_i is the i-th column of U.

$$\|u_i\| = 1$$

Each column has unit length.

Inner product

$$u_i^T u_j = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$$

$$U^T U = I_m$$

I_m is the m × m identity matrix.

The columns of V

$$V \in \mathbb{R}^{n \times n}$$

$$V = [v_1, v_2, \ldots, v_n]$$

v_i is the i-th column of V.

$$\|v_i\| = 1$$

Each column has unit length.

Inner product

$$v_i^T v_j = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$$

$$V^T V = I_n$$

I_n is the n × n identity matrix.

The matrix Σ

$$\Sigma \in \mathbb{R}^{m \times n}, \quad p = \min(m, n)$$

$$\Sigma_{ij} = \begin{cases} \sigma_i & i = j \\ 0 & i \neq j \end{cases}$$

Matrix form when m = 3 and n = 4

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & 0 & 0 \\ 0 & \sigma_2 & 0 & 0 \\ 0 & 0 & \sigma_3 & 0 \end{bmatrix}$$

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0$$

$\sigma_1, \ldots, \sigma_p$ are the singular values.

Singular values: a numerical example

A simple example

$$M = \begin{bmatrix} 10 & 0 \\ 0 & 4 \end{bmatrix} \qquad U = V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad \Sigma = \begin{bmatrix} 10 & 0 \\ 0 & 4 \end{bmatrix}$$

$$Mv_1 = \begin{bmatrix} 10 \\ 0 \end{bmatrix} = 10u_1 \qquad \sigma_1 = 10$$

$$Mv_2 = \begin{bmatrix} 0 \\ 4 \end{bmatrix} = 4u_2 \qquad \sigma_2 = 4$$

Both input vectors have length 1. M stretches them to lengths 10 and 4.

For k = 1, keep v_1, paired with $\sigma_1 = 10$ (and u_1).

Larger singular value: more weight in reconstruction, not guaranteed better retrieval.

Two matrices: A and B

Let A and B be two m × n patch matrices.

For each matrix, select the V columns paired with its k largest singular values.

$$A = U_A \, \Sigma_A \, V_A^{\mathrm{T}}$$

$$B = U_B \, \Sigma_B \, V_B^{\mathrm{T}}$$





$$P = [v_{A,1}, \ldots, v_{A,k}]$$

$$Q = [v_{B,1}, \ldots, v_{B,k}]$$

$$P, Q \in \mathbb{R}^{\mathrm{n} \times \mathrm{k}}$$

Cross-Gram matrix H

$$H = P^{\mathrm{T}} Q \in \mathbb{R}^{k \times k}$$

$$H_{ij} = p_i^{\mathrm{T}} \, q_j$$

H need not be diagonal or equal to the identity matrix.

Apply SVD to H and record its singular values.

$$H = U_H \, \Sigma_H \, V_H^{\mathrm{T}}$$

$$\downarrow \qquad \downarrow \qquad \downarrow$$

$$H = L \, \mathrm{diag}(s_1, \ldots, s_k) \, R^{\mathrm{T}}$$

$s_1, \ldots, s_k$ are the singular values of H.

$$1 \geq s_1 \geq \cdots \geq s_k \geq 0$$

Grassmann geodesic distance

Use the singular values of H to obtain the principal angles.

$$\theta_j = \arccos(s_j) \quad \rightarrow \quad d = \sqrt{\theta_1^2 + \cdots + \theta_k^2}$$

Angles are in radians.

$d = 0$ The selected subspaces are the same.
The images need not be identical.

Smaller d means closer subspaces.

Larger d means farther subspaces. At fixed k, rank smaller distances first.

Source of the distance

The Geometry of Algorithms with Orthogonality Constraints

Alan Edelman, Tomás A. Arias and Steven T. Smith (1998)

Read Section 4.3, page 337

They study optimization with orthogonality constraints.

This distance measures the shortest path length between subspaces in the standard Grassmann geometry.

Changing the basis within either subspace leaves d unchanged.

A common patch grid

The saved images have different patch-grid sizes.

We reuse the saved features. We do not run DINO again.

Saved full SVD

$$M = U\Sigma V^T$$

Bilinear resampling

14 × 14 patch grid

$$M^*$$

196 × 384

All five use M*. Cases 2–5 use its SVD.

A common grid size does not guarantee that objects line up.

Five experiment configurations

One matrix baseline and four subspace comparisons.

Case	Data used	Distance
1	Full M*	RMSE
2	U: top 32 directions	Geodesic
3	V: top 8 directions	Geodesic
4	V: top 16 directions	Geodesic
5	V: top 32 directions	Geodesic

8, 16 and 32 are exploratory choices. We have not selected an optimal rank.

$$\mathrm{RMSE} = \frac{\| M_{\mathrm{Q}}^{*} - M_{\mathrm{I}}^{*} \|_{\mathrm{F}}}{\sqrt{(196 \times 384)}}$$

mAP: scoring the ranking

Example: two correct matches, found at ranks 1 and 3.

Precision = correct matches found ÷ results seen.

Precision before → after	**Rank 1**	**Rank 2**	**Rank 3**
	Correct	Incorrect	Correct
	$1 \to 1$	—	$\frac{1}{2} \to \frac{2}{3}$

Average precision before and after each correct hit (start at 1).

Multiply each average by ½: each hit finds one of the two targets.

$$\mathrm{AP} = \frac{1}{2} \times \frac{1+1}{2} + \frac{1}{2} \times \frac{\frac{1}{2}+\frac{2}{3}}{2} \approx 79.17\%$$

$$\mathrm{mAP} = \frac{\mathrm{AP}_1 + \cdots + \mathrm{AP}_{70}}{70}$$

Average over 70 queries. Higher is better.

Exploratory retrieval results

70 queries, 4,993 gallery images. The five new cases use M*.

Representation	Medium mAP (%)	Hard mAP (%)
CLS reference	32.25	7.82
1. Full M* (RMSE)	9.14	2.02
2. U, top 32	4.57	0.82
3. V, top 8	19.54	3.53
4. V, top 16	25.91	5.98
5. V, top 32	25.73	5.56

Higher mAP is better. mAP evaluates rankings, not ordinary accuracy.

All five trail CLS. V16 leads the five, ahead of V32.

Nine Steps

Steps 1–6

1 Dataset

1 choice

A ROxford

70 queries
4,993 gallery images

2 Image size

3 choices

A 224 pixels

B 320 pixels

C 448 pixels

Short edge
16 × 16 pixel patches

3 CLS token

1 choice

A Original learned token

Kept fixed

4 Transformer

1 choice

A DINO ViT-S/16

Frozen weights

5 Patch matrix

4 choices

A Raw
B Subtract patch mean
C Unit-length patches
D Common 14 × 14 grid

6 Decomposition

1 choice

A SVD

Full-M control uses
no SVD factors.

Nine Steps

7 Distance

8 settings

- **A** Projection
- **B** Geodesic
- **C** Procrustes
- **D** Spectral
- **E** Fubini–Study
- **F** Bures: σ weights
- **G** Bures: σ² weights
- **H** RMSE

8 Directions / full M

15 choices

- **A** V: k = 1
- **B** V: k = 2
- **C** V: k = 4
- **D** V: k = 8
- **E** V: k = 16
- **F** V: k = 32
- **G** V: k = 64
- **H** U: k = 1
- **I** U: k = 2
- **J** U: k = 4
- **K** U: k = 8
- **L** U: k = 16
- **M** U: k = 32
- **N** U: k = 64
- **O** Full M

9 Evaluation

1 fixed rule

- **A** Medium + Hard mAP

Numbers identify the steps. Letters identify the choices within each step.

Nine Steps

Three lock wheels, three choices each: $3 \times 3 \times 3 = 27$

Our choices must fit together. Count the three groups separately.

	1 Dataset		2 Size		3 CLS		4 Model		5 Matrix		6 SVD		7 Distance		8 Directions		9 Score		
V	1	×	3	×	1	×	1	×	4	×	1	×	7	×	7	×	1	=	**588**
U	1	×	3	×	1	×	1	×	1	×	1	×	5	×	7	×	1	=	**105**
Full M	1	×	3	×	1	×	1	×	1	×	1	×	1	×	1	×	1	=	**3**

U and full M use matrix D. Distances: V uses A–G, U uses A–E, full M uses H.

Combinations $588 + 105 + 3 = 696$

600 scored
84 equivalent, 12 invalid

Top Five

Steps 1–9 use the letter choices on the previous pages.

Example: step 2, choice C = 448 pixels.

No.	1	2	3	4	5	6	7	8	9	mAP (%) Medium	Hard
1	**A**	**A**	**A**	**A**	**A**	**A**	**A**	**F**	**A**	**27.93**	**6.64**
2	A	A	A	A	B	A	A	F	A	27.86	6.57
3	A	C	A	A	B	A	A	F	A	27.65	5.11
4	A	A	A	A	D	A	A	F	A	27.60	6.52
5	A	B	A	A	B	A	A	E	A	27.52	6.10
CLS reference: 224										32.25	7.82
CLS reference: 320										31.47	8.67
CLS reference: 448										31.22	8.27

Ranked by Medium mAP. None beats its same-size CLS reference.

Grassmann Pooling as Compact Homogeneous Bilinear Pooling for Fine-Grained Visual Classification

Wei et al., ECCV 2018

1	2	3	4	5	6
CNN features	**Reduce dimension**	**Matrix A**	**SVD → U_k**	**Compare with W_i**	**Predict the class**
VGG-16	512 → 256 / 64	Vectors as columns	Keep k = 16	Projection score	Highest score

The training images have class labels.

Source: Fig. 4 and §3

5. Image and class matrices



Input image

CNN + reduction + matrix + SVD

$$U_k = [u_1, \ldots u_k]$$

k directions for this image



Class label: Elegant Tern

Training with labeled images

$$W_i$$

Learned matrix for class i

After training: W_i stays fixed. U_k changes with each input image.

Source: §3.3 and Fig. 1. Pictures illustrate the workflow.

5. Projection score

1 Multiply the two matrices $H_i = W_i^{\mathrm{T}} U_k$

2 Square every entry, then add $\mathrm{score}_i = \|H_i\|_F^2$

Illustration: k = 1, with made-up numbers

Two matrices

$$U_k = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad W_i = \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix}$$

Matrix product

$$0.8 \times 1 + 0.6 \times 0 = 0.8$$

Score

$$0.8^2 = 0.64$$

Compute one score per class. The highest score gives the prediction.

Source: §3.3. A score is not a probability.